CP-SAT Formulation

Technical Deep Dive for Data Science

Integer Programming | Constraint Satisfaction | Google OR-Tools

model.Add(solver.ResponseProto)

1. Sets, Indices & Decision Variables

SETS & INDICES

$$D = \{0, 1, 2, 3, 4\}$$

Set of production days (Mon–Fri), where $|D| = N_D = 5$.

$$M = \{m_1, m_2, \ldots, m_k\}$$

Set of unique truck models derived from demand rows.

$$L = \{\ell_1, \ell_2, \ell_3\}$$

Set of production lines (Line Sorts).

$$q_{m,\ell} \in \mathbb{Z}_{\geq 0}$$

Input parameter: Required quantity for model m on line ℓ.

DECISION VARIABLES

$$x_{m,\ell,d} \in \{0, 1, \ldots, q_{m,\ell}\}$$

Primary Integer Variable
Number of units of model m assigned to line ℓ on day d.

$$y_{m,d} = \sum_{\ell \in L} x_{m,\ell,d}$$

Derived Variable
Total daily production of model m across all lines.

$$z_{\ell,d} = \sum_{m \in M} x_{m,\ell,d}$$

Derived Variable
Total daily workload on line ℓ.

2. Hard Constraints Formulation

1. Demand Satisfaction

$$\sum_{d \in D} x_{m,\ell,d} = q_{m,\ell} \quad \forall (m, \ell)$$

Ensures that the total units assigned across all 5 days exactly match the required quantity for every model-line pair. No overproduction, no shortages.

```
model.Add(
  sum(x[m,l,d] for d in D)
  == q[m,l]
)
```

2. Small-Model Spread

$$\lfloor \frac{Q_m}{N_D} \rfloor \leq y_{m,d} \leq \lceil \frac{Q_m}{N_D} \rceil$$

For low-volume models ($Q_m < T$), forces daily production to be within the floor and ceiling of the daily average. Prevents "clumping" of small batches.

```
avg = Q_m // 5
model.Add(y[m,d] ≥ avg)
model.Add(y[m,d] ≤ avg + 1)
```

3. Capacity Caps

$$\sum_{(m,\ell) \in S_{2512}} x_{m,\ell,d} \leq 1 \quad \forall d$$

Global daily limit for specific constraint groups (e.g., Rear Loader 2512). Ensures no more than 1 unit of this type is scheduled per day across all lines.

```
model.Add(
  sum(x[..] for .. in S_2512)
  ≤ 1
)
```

3. Soft Constraints & Penalty Logic

THE BUSINESS RULE

Certain models have "Preferred Days" (e.g., Tue/Thu). We want to schedule them there if possible, but allow deviations if necessary to maintain overall balance.

THE PENALTY VARIABLE

We introduce a binary variable $p_{m,\ell}$ that acts as a "flag":

- $p = 0$: Rule Satisfied (Good)
- $p = 1$: Rule Violated (Penalty)

PREFERRED SET

Let $P \subset D$ be the set of preferred days.
Example: $P = \{1, 3\}$ for Tuesday & Thursday.

$$p_{m,\ell} = \begin{cases} 0 & \text{if } \sum_{d \in P} x_{m,\ell,d} \geq 1 \\ 1 & \text{otherwise} \end{cases}$$

```
# CP-SAT Implementation

# 1. Create Boolean 'is_satisfied'
is_satisfied = model.NewBoolVar(name)

# 2. Link production to satisfaction
# If sum(x on pref days) ≥ 1, then is_satisfied can be
true
model.Add(
  sum(x[m,l,d] for d in P) ≥ 1
).OnlyEnforceIf(is_satisfied)

# 3. Define Penalty (p = NOT is_satisfied)
# In objective: Minimize W * (1 - is_satisfied)
```

4. Objective Function & Linearization

MINIMIZE TOTAL COST (Z)

$$\min Z = \alpha \sum_{m,d} u_{m,d} + \beta \sum_{\ell,d} v_{\ell,d} + W \sum_{(m,\ell)\in\mathcal{P}} p_{m,\ell}$$

$\alpha \cdot u_{m,d}$

Model Imbalance
Penalizes deviation from ideal daily average for each model.

$\beta \cdot v_{\ell,d}$

Line Imbalance
Penalizes deviation from ideal daily workload for each line.

$W \cdot p_{m,\ell}$

Preference Penalty
Cost incurred when soft constraints (Tue/Thu, Fri) are violated.

Linearization of Absolute Deviation

The Problem:
We want to minimize $|N_D \cdot y_{m,d} - Q_m|$.
Absolute value is non-linear.

The CP-SAT Solution:
Introduce auxiliary variable $u_{m,d} \geq 0$ and constraints:

$$u_{m,d} \geq (N_D \cdot y_{m,d} - Q_m)$$

$$u_{m,d} \geq -(N_D \cdot y_{m,d} - Q_m)$$

5. Solver Configuration & Search Strategy

Presolve Phase

Aggressively simplifies the model before search begins. Detects fixed variables, merges constraints, and reduces domain sizes to shrink the search space.

Parallel Search

Utilizes deterministic parallel search (8 workers default). Different workers try different heuristics (LNS, Core-based) simultaneously to escape local optima.

Time Budgeting

Enforces strict runtime limits (e.g., 30s) for operational responsiveness. Returns the best feasible solution found if optimality isn't proven within the limit.

```
# Configure CP-SAT Solver
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 30.0
solver.parameters.num_search_workers = 8
solver.parameters.log_search_progress = True

$ python3 scheduler.py
[INFO] Starting CP-SAT solver...
[INFO] Parameters: time_limit=30s, workers=8
[LOG] Presolve removed 15 variables and 8 constraints.
[LOG] Search worker 0: found solution obj=1250
[LOG] Search worker 3: found solution obj=840
[LOG] Search worker 1: found solution obj=420
[DONE] Optimal solution found.
[DONE] Objective value: 420
[DONE] Wall time: 2.45s
```